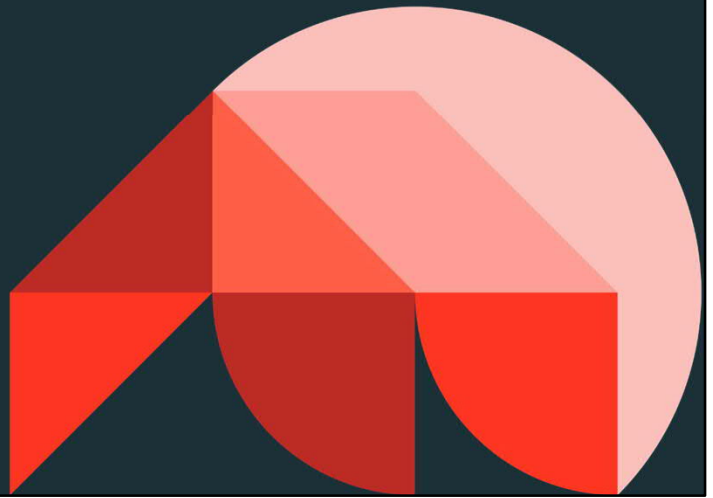




Spark Structured Streaming



©2024 Databricks Inc. — All rights reserved



Agenda topics

Introduction to Streaming
Streaming Data Concepts
Introduction to Structured Streaming
Streaming from Delta Lake





Introduction to Streaming

LECTURE

Streaming Data Concepts



©2024 Databricks Inc. — All rights reserved



Stream Processing

Why is stream processing getting popular?

Data Velocity & Volumes

Rising data velocity & volumes requires continuous, incremental processing - cannot process all data in one batch on a schedule

Real-time analytics

Businesses demand access to fresh data for actionable insights and faster, better business decisions

Operational applications

Critical applications need real-time data for effective, instantaneous response



The vast majority of the data in the world is streaming data!

©2024 Databricks Inc. — All rights reserved



Stream Processing Use Cases

Stream processing is a key component of big data applications across all industries



Notifications



Real-time reporting



Incremental ETL



Update data to serve in
real-time



Real-time decision making



Online ML

©2024 Databricks Inc. — All rights reserved



Advantages of Stream Processing

Why use streaming (vs. batch) ?



Incremental processing of new data is natural and intuitive



Unlocks low latency SLAs for time sensitive workloads



Better fault-tolerance upon restarts

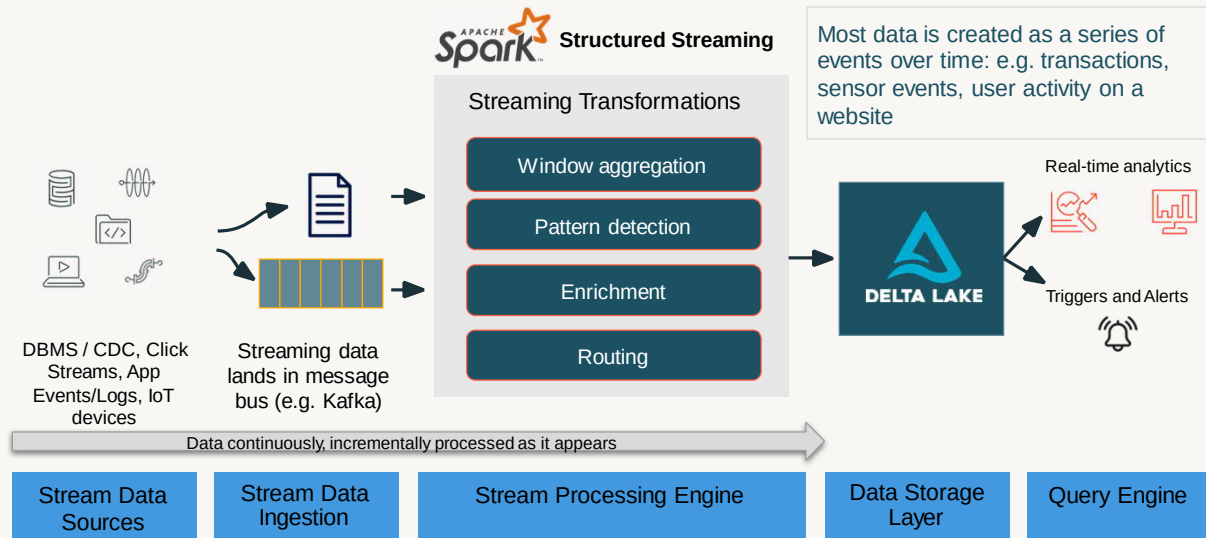


Higher compute utilization and scalability through continuous and incremental processing



Stream Processing Architecture

Modern data processing pipeline



©2024 Databricks Inc. — All rights reserved



Challenges of Stream Processing

Stream processing is not easy

- Processing each event exactly once despite machine failures
- Processing out-of-order data based on application timestamps (also called event time)
- Maintaining large amounts of state
- Handling load imbalance and stragglers
- Determining how to update output sinks as new events arrive
- Writing data transactionally to output systems



©2024 Databricks Inc. — All rights reserved



A **straggler** is an event (or task) in a distributed stream-processing pipeline that **falls behind the expected processing time**. Even if most tasks finish quickly, a few slow ones can delay the overall system because stream processors often need to wait for all tasks or partitions to finish before triggering:

- watermarks
- window computations
- joins
- aggregations

This leads to **increased latency, out-of-order results, or backpressure**.



Introduction to Streaming

LECTURE

Introduction to Structured Streaming



©2024 Databricks Inc. — All rights reserved



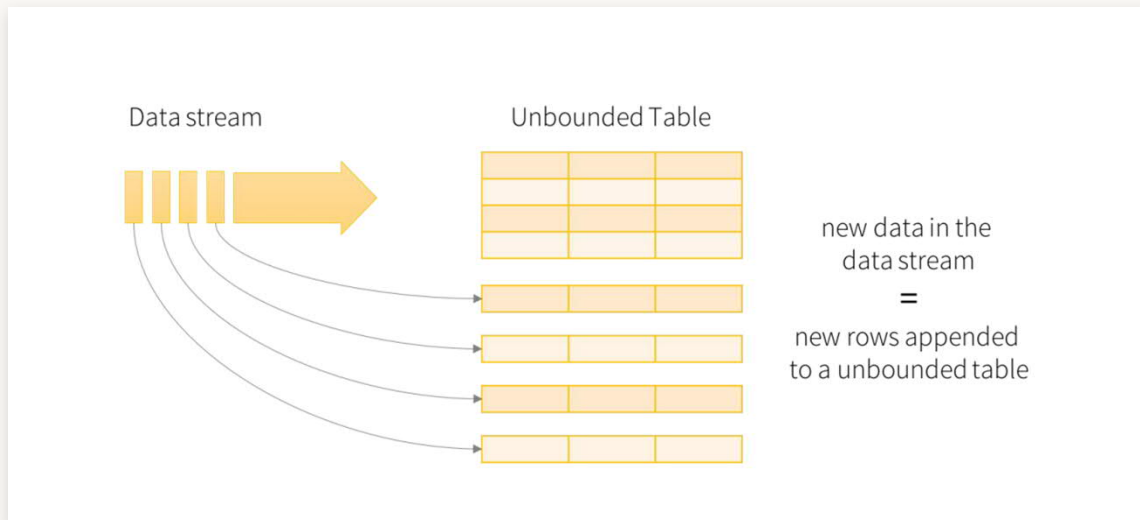
What is Structured Streaming

Apache Spark structured streaming basics

- A scalable, fault-tolerant **stream processing framework** built on Spark SQL engine.
- Uses **existing structured APIs** (DataFrames, SQL Engine) and provides similar API as batch processing API.
- Includes **stream specific features**; end-to-end, exactly-once processing, fault-tolerance etc.



Streams as Unbounded Tables



©2024 Databricks Inc. — All rights reserved



In **Spark Structured Streaming**, an **unbounded table** is a mental model used to represent a continuous stream of data. Spark treats a data stream **as if it were a table that is continuously growing over time**, with new rows appended as events arrive.

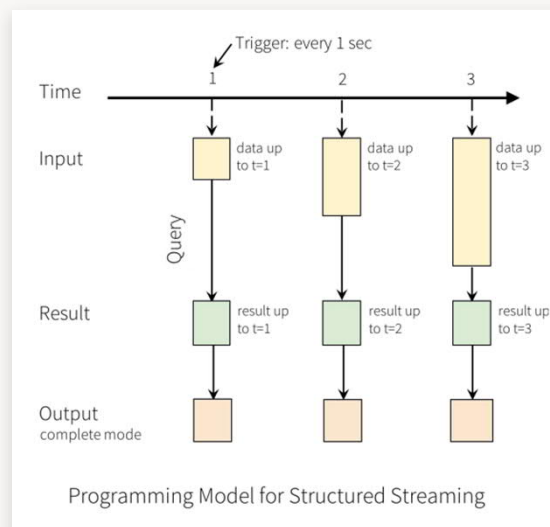
An **unbounded table** is:

- A logical table that **never stops growing**
- New data keeps arriving continuously
- There is **no predefined end**, unlike a static/batch table
- Queries run **incrementally**, updating results as new rows appear

How Structured Streaming Works

Execution mode

1. An **input table** is defined by configuring a streaming read against **source**.
2. A **query** is defined against the input table.
3. This logical query on the input table generates the **results table**.
4. The **output** of a streaming pipeline will persist updates to the results table by writing to an external **sink**.
5. New rows are appended to the input table for each **trigger interval**.



©2024 Databricks Inc. — All rights reserved



Behind the scenes, Spark runs a **trigger loop** (micro-batch or continuous) that does:

- Check the source (Kafka, Delta, files) for *new* data
- Fetch that new data
- Treat it as a **new set of rows appended to the unbounded table**
- Apply your query logic (transformations, joins, aggregations)
- Write the results to the sink
- Update offsets/checkpoints
- Sleep → then repeat

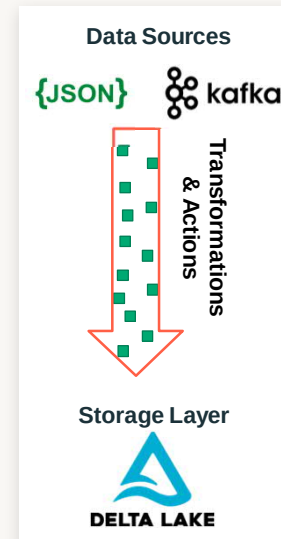
In **Spark Structured Streaming**, when you use **files as the input source** (JSON, CSV, Parquet, ORC, text), Spark reads **incremental batches of new files** instead of re-scanning the entire directory.

- Spark Watches a Directory, Not the Files Themselves
- Perform incremental Discovery via File Listing

Anatomy of a Streaming Query

Structured streaming core concepts

- Example:
 - Read JSON data from Kafka
 - Parse nested JSON
 - Store in structured Delta Lake table
- Core concepts:
 - Input sources
 - Sinks
 - Transformations & actions
 - Triggers



©2024 Databricks Inc. — All rights reserved



Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")  
  .option("kafka.bootstrap.servers", "...")  
  .option("subscribe", "topic")  
  .load()
```

Returns a Spark DataFrame
(common API for batch & streaming data)

Source:

- Specify where to read data from
- OS Spark supports Kafka and file sources
- Databricks runtimes include connector libraries supporting Delta, Event Hubs, and Kinesis

```
spark.readStream.format(<source>)  
  .option(<>, <>)...  
  .load()
```



Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", ...)
  .option("subscribe", "topic")
  .load()
  .selectExpr("cast (value as string) as json")
  .select(from_json("json", schema).as("data"))
```

Transformations:

- 100s of built-in, optimized SQL functions like `from_json`
- In this example, cast bytes from Kafka records to a string, parse it as JSON, and generate nested columns



Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", ...)
  .option("subscribe", "topic")
  .load()
  .selectExpr("cast (value as string) as json")
  .select(from_json("json", schema).as("data"))
  .writeStream
    .format("delta")
    .option("path", "/deltaTable/")
```

Sink: Write transformed output to external storage systems

Spark includes support for Delta as both streaming source and sink.

OS Spark supports:

- Files and Kafka for production
- Console and memory for development and debugging
- Many open source connectors available for Enterprise platforms including Azure & AWS Streaming services, and Databases, etc.

©2024 Databricks Inc. — All rights reserved



There **are** several open-source connectors for **OSS (open-source) Apache Spark Structured Streaming** besides the built-in **file** and **Kafka** sources.

Amazon provides an **open-source Spark Structured Streaming connector for Kinesis**.

- It is **fully open source**, built on Spark's DataSource V2 API.
- Supports both provisioning modes and enhanced fan-out.
- Distributed as a standalone JAR for any Spark deployment.

Microsoft publishes an **open-source Event Hubs connector** on GitHub.

- Supports both Spark Streaming and **Structured Streaming**.
- Licensed under Apache-2.0.
- Works on OSS Spark (not Databricks-specific).

Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", ...)
  .option("subscribe", "topic")
  .load()
  .selectExpr("cast (value as string) as json")
  .select(from_json("json", schema).as("data"))
  .writeStream
    .format("delta")
    .option("path", "/deltaTable/")
    .trigger("1 minute")
    .option("checkpointLocation", "...")
  .start()
```

- **Checkpoint location:** For tracking the progress of the query
- **Output Mode:** Defines how the data is written to the sink; Equivalent to “save” mode on static DataFrames
- **Trigger:** Defines how frequently the input table is checked for new data; Each time a trigger fires, Sparks check for new data and updates the results



DataFrame as a Unifying API

Batch

```
spark
  .read
    .table("source_name")
    .withColumn(...)
    .select(...)
  .write
    [ .outputMode(...) ]
    .saveAsTable("sink_table")
```

Streaming

```
spark
  .readStream
    [ .withWatermark(...) ]
    .table("source_name")
    .withColumn(...)
    .select(...)
  .writeStream
    [ .trigger(...) ]
    [ .queryName(...) ]
    .option("checkpointLocation", ...)
    .toTable("sink_table")
```

©2024 Databricks Inc. — All rights reserved



`withWatermark()` is a **Structured Streaming** API feature in Spark that tells the engine **how long to wait for late events based on event-time**, and **when it can safely drop old state** from memory. A watermark defines a “**lateness threshold**” for event-time data.

Anatomy of a Streaming Query

Structured streaming core concepts

```
val df = spark.readStream
  .format("json")
  .load("path")
  .withWatermark("eventTime", "10 minutes")
```

User	eventTime (when click happened)	processingTime (when Spark receives it)
A	12:00	12:00 (on time)
B	12:02	12:02 (on time)
C	12:01	12:10 (late)

Anatomy of a Streaming Query

Structured streaming core concepts

Trigger Types:

Fixed interval micro batch	<code>.trigger(processingTime = "2 minutes")</code>	Micro-batch processing kicked off at the <i>user-specified interval</i>
Triggered One-time micro batch	<code>.trigger(once=True)</code>	Process all of the available data as a <i>single micro-batch</i> and then automatically stop the query
Triggered One-time micro batches	<code>.trigger(availableNow=True)</code>	Process all of the available data as <i>multiple micro-batches</i> and then automatically stop the query
Continuous Processing	<code>.trigger(continuous= "2 seconds")</code>	EXPERIMENTAL - Long-running tasks that <i>continuously</i> read, process, and write data as soon events are available, with checkpoints at the specified frequency
Default		Databricks: 500ms fixed interval OS Apache Spark: Process each microbatch as soon as the previous has been processed



Anatomy of a Streaming Query

Structured Streaming Core Concepts

Output Modes:

Complete	• The entire updated Result Table is written to the sink. • The individual sink implementation decides how to handle writing the entire table.
Append	Only the new rows appended to the Result Table since the last trigger are written to the sink.
Update	Only new rows and the rows in the Result Table that were updated since the last trigger will be outputted to the sink.

Note: The output modes supported depends on the type of [transformations and sinks used by the streaming query](#).





Introduction to Streaming

Streaming from Delta Lake



©2024 Databricks Inc. — All rights reserved



Streaming from Delta Lake

Using a Delta table as a streaming source

- Each committed version represents new data to stream. Delta Lake transactions logs identify the version's new data files
- Structured Streaming assumes append-only sources. Any non-append changes to a Delta table causes queries streaming from that table to throw exceptions.
 - Set `delta.appendOnly = true` to prevent non-append modifications to a table.
 - Use Delta Lake [change data feed](#) to propagate arbitrary change events to downstream consumers

©2024 Databricks Inc. — All rights reserved



Delta Lake can enforce append-only behavior using table properties:

```
ALTER TABLE tbl
SET TBLPROPERTIES ('delta.appendOnly' = 'true');
```

When `delta.appendOnly = true`:

- INSERTS are allowed
- UPDATES fail
- DELETES fail
- OVERWRITES fail

Example from Delta Internals:

Trying to DELETE from an append-only table throws:

[DELTA_CANNOT_MODIFY_APPEND_ONLY] This table is configured to only allow appends.

Append-only tables are common when:

- ingesting immutable log files
- IoT data events
- clickstream events
- financial tick data
- CDC streams where only inserts are kept (raw bronze layer)
- These tables grow monotonically, without mutation.

Streaming from Delta Lake

Using a Delta table as a streaming source

- You can limit the input rate for micro-batches by setting DataStreamReader options:
 - maxFilesPerTrigger: Maximum files read per micro-batch (default 1,000)
 - maxBytesPerTrigger: Soft limit to amount of data read per micro-batch (no default)



Streaming to Delta Lake

Using a Delta table as a streaming sink

- Each micro-batch written to the Delta table is committed as a new version.
- Delta Lake supports both append and complete output modes.
 - Append is most common.
 - Complete replaces the entire table with each micro-batch. It can be used for streaming queries that perform arbitrary aggregations on streaming data.

